C语言运算符详解

全面掌握C语言的运算与操作

1. 算术操作符：+、-、*、/、%

算术操作符概述

C语言提供了五种基本算术运算符：加法（+）、减法（-）、乘法（*）、除法（/）、取模（%）。这些运算符用于执行基本的数学计算。

1.1 + 和 -

+ 和 - 也可作为单目运算符使用。

```
int a = 5;
int b = -a;  // b 现在是 -5
int c = +a;  // c 现在是 5
```

1.2 *

运算符**两侧**必须是数值。

```
int a = 10;
int b = 20;
int c = a * b;  // c 现在是 200
```

1.3 /

运算符两侧必须是数值。

除法结果取决于操作数的类型：

- 两个整数进行整除，会丢弃小数部分。
- 至少一个浮点数则进行浮点除法，保留小数部分。

```
int a = 7;
int b = 2;
int c = a / b;  // c 现在是 3 (整除)

float x = 7.0;
int y = x / b;  // y 现在是 3.500000
int z = a / 2.0;  // z 现在是 3.500000
```

上面示例中，尽管变量a的类型是int（浮点数），但除以4的结果是3.5，而不是1.5。原因就在于：**进行除法运算的两个操作数，其中至少有一个是浮点数，则会执行浮点除法，否则会执行整除。**

如果希望强迫两个操作数进行浮点除法，**则可以将其中一个或两个操作数，通过（）进行类型转换**。

```
int a = 7;
int b = 2;
int c = (float)a / b;  // 结果为 3.500000
int d = a / (float)b;  // 结果为 3.500000
```

上面示例中，5.0 / 4的结果仍旧是浮点数运算，得到的结果是1.25。

1.4 %

运算符%表示求模/余数，即返回两个整数相除的余数。这个运算符只能用于整数，不能用于浮点数。

```
int a = 10;
int b = 3;
int c = a % b;  // c 现在是 1
int d = 5 % 5;  // d 现在是 0
int e = 7 % 2;  // e 现在是 1
```

2. 赋值操作符：=和复合赋值

赋值操作符概述

简单赋值通过使用等号（=）来实现。其语法为：变量 = 表达式。

```
int a = 10;
a = a + 5;
a = a * 2;
```

C语言提供多种便捷的复合赋值操作符，可以将算术运算与赋值结合起来。

```
int a = 10;
a += 5;  // 等价于 a = a + 5
a *= 2;  // 等价于 a = a * 2
```

这些复合赋值操作符等价于更长的写法：

```
int a = 10;
a += 5;
a *= 2;
```

C语言中的扩展赋值操作，共有如下几种：

```
+=    -=    *=
/=    %=    <<=
>>=   &=    ^=
|=
```

3. 增量操作符：++、--

增量操作符概述

前缀++和前缀--操作数前面的操作符，称为增量操作符。

3.1 ++ 和 --

++是一种增量操作符，又称为前置++和后置++，统称为自增操作符，也称为递增-和后置--，称为自减操作符。

3.1.1 前置++

```
int a = 10;
int b = ++a;  // 先执行 a = a + 1，然后将 a 的值赋给 b，现在 a 和 b 都是 11
printf("a=%d b=%d\n", a, b);
```

```
计算结果：a=11, b=11
```

3.1.2 后置++

```
int a = 10;
int b = a++;  // 先将 a 的值赋给 b，然后执行 a = a + 1，现在 a 是 11，b 是 10
printf("a=%d b=%d\n", a, b);
```

```
计算结果：a=11, b=10
```

3.1.3 前置--

```
int a = 10;
int b = --a;  // 先执行 a = a - 1，然后将 a 的值赋给 b，现在 a 和 b 都是 9
printf("a=%d b=%d\n", a, b);
```

```
计算结果：a=9, b=9
```

3.1.4 后置--

```
int a = 10;
int b = a--;  // 先将 a 的值赋给 b，然后执行 a = a - 1，现在 a 是 9，b 是 10
printf("a=%d b=%d\n", a, b);
```

```
计算结果：a=9, b=10
```

总结：

- 如果你想了解更多——前置、后置、自增、自减的区别，请查阅C语言教程。
- 如果你想了解更多——前置++、后置++、前置--、后置--的区别，请查阅C语言教程。

4. 移位运算符

左移操作符：将左边的数，向左移动N位

```
int num = 20;
int n = 3;
printf("a = %d", a);
printf("num = %d", num);
```

右移操作符：

小结：关于移位操作的注意事项

- 位移运算符的操作数可以是整数或字符型数据，但不能是浮点数或复杂类型（如结构体）。
- 位移运算符的右操作数，必须是一个非负整数，且其值不应超过当前数据类型的位数。
- 编译器可以对移位操作进行优化，例如将乘以2的幂次方的运算转换为左移，或将除以2的幂次方的运算转换为右移。

4.2 右移操作符

右移操作：将左边的数，向右移动N位

```
int num = 20;
int n = num >> 3;
printf("a = %d", a);
printf("num = %d", num);
```

此时打印结果，即等价于：结果为：2

```
int num = 20;
int n = num >> 2;
printf("a = %d", a);
printf("num = %d", num);
```

此时打印结果，即等价于：结果为的数据：5

5. 位运算符

位运算符概述

&	位按位与
|	位按位或
^	位按位异或
~	位按位取反

```
int num = 20;
int n = 3;
printf("a = %d", a);
printf("num = %d", num);
printf("num1 = %d", num1);
printf("num2 = %d", num2);
printf("num3 = %d", num3);
printf("num4 = %d", num4);
```

示例：如何代码实现：求一个整数存储在内存中的二进制中1的个数。

这是一个经典的面试题，我们来看三种不同的解法。

第1种：逐位检查

```
int num = 20;
int n = 3;
int count = 0;
while (num != 0)
{
    if (num & 1) // num & 1 检查最低位
    {
        count++;
    }
    num = num >> 1; // num = num >> 1
}
printf("二进制中1的个数 = %d", count);
return 0;
}
```

思考：这种实现方式是否有问题？

第2种：固定循环次数法

```
int num = 20;
int n = 3;
int count = 0;
for (int i = 0; i < 32; i++) // 假设是32位整数
{
    if (1 & num >> i)
    {
        count++;
    }
}
printf("二进制中1的个数 = %d", count);
return 0;
}
```

思考：这是不是更加合理，因为它能解决以前的问题？

第3种：布莱恩科尔算法

```
int num = 20;
int n = 3;
int count = 0;
while (num != 0)
{
    count++;
    num = num & (num - 1); // 这个操作会消除最后一位1
}
printf("二进制中1的个数 = %d", count);
return 0;
}
```

思考：这种方法非常巧妙，它是如何工作的？

6. 关系运算符

关系运算符

关系运算符用于比较两个数值，结果为真或假，则结果为1，假则为0。

```
int a = 15;
int b = 10;
printf("a > b : %d\n", a > b);  // 输出结果 1 (真)
printf("a < b : %d\n", a < b);  // 输出结果 0 (假)
printf("a >= b : %d\n", a >= b);
printf("a <= b : %d\n", a <= b);
printf("a == b : %d\n", a == b);
printf("a != b : %d\n", a != b);
return 0;
}
```

7. 逻辑运算符

逻辑运算符

逻辑运算符：逻辑与 &&、逻辑或 ||、逻辑非 !。

这些运算符通常用作关系运算符的结合使用，用于构建更复杂的条件判断。例如，我们可以同时检查两个数字是否都大于0。

8. 逗号表达式

逗号表达式概述

逗号表达式，就是用逗号将两个表达式，连接起来的表达式。

逗号表达式会从左向右执行其所有子表达式，整个表达式的最终结果是最后一个子表达式的结果。

```
int a = 1;
int b = 2;
int c = (a = 5, b = 8, a + b);  // 执行顺序是从左到右
printf("c = %d\n", c);
return 0;
```

这段简单的一段代码，展示了逗号表达式的执行顺序。

本节总结

- **算术操作符**：+、-、*、/、%，与数学中的运算规则基本保持一致。
- **赋值操作符**：=，用来将右侧的值赋给左侧的变量。
- **复合赋值**：+=、-=、*=、/=、%=，用于简化代码，例如 a += 5 等价于 a = a + 5。
- **增量操作符**：++（自增）和 --（自减），以及它们的变体。
- **移位操作符**：<<（左移）和 >>（右移），用于位运算，例如 num << 3 表示将 num 的二进制向左移动 3 位。
- **位运算符**：&（按位与）、|（按位或）、^（按位异或）、~（按位取反）。
- **关系运算符**：>、<、>=、<=、==、!=，用于比较两个数值的关系，结果为真（1）或假（0）。
- **逻辑运算符**：&&（逻辑与）、||（逻辑或）、!（逻辑非），用于构建复杂的条件判断。
- **逗号表达式**：,，用于将多个表达式连接起来，整个表达式的结果是最后一个子表达式的结果。